COS30019- Introduction to AI

Assignment 2B

Name	Student ID
Neng Yi Chieng	104386364
Ivan Liang Jin Ngu	104381576
Muzamil Khan Jakhro	102789369

Contents

Instruction 4
System requirement 4
Installation 4
Run the program 4
Using the System Interface 4
Introduction 6
Features 7
Route Setup and Configuration 7
Traffic Incident Reporting and Safety Analysis 7
AI-Based Incident Classification 7
Route Calculation and Visualization 8
Step-by-Step Route Animation 8
Bugs 8
Missing or Not Implemented Features 8
Testing 9
Test Case 1: Valid Route Configuration and Successful Route Calculation 9
Test Case 2: Input Validation for Invalid Route Configuration 9
Test Case 3: Route Visualization and Step-by-Step Animation 9
Test Case 4: Manual Traffic Incident Reporting and Route Impact 10
Test Case 5: AI-Based Incident Classification and Severity Detection 10
Testing Summary 10
Insights 11
Conclusion 12
Appendix 13
Appendix A – User Interface 13
Appendix B- AI Model Training 16
Appendix C- Testing 20
Acknowledgements 23

References 24

Figure 1:Route Setup Interface 13
Figure 2:Safety Analysis – Manual Incident Reporting 13
Figure 3: AI Incident Classification Interface 14
Figure 4: AI Incident Classification Result Display 14
Figure 5: Route Calculation and Visualization Interface 15
Figure 6: Route Playback and Animation Controls 15
Figure 7:Calculated Route Preview and Visualization 16
Figure 8:Learning Graph EfficientNet 16
Figure 9:Confusion Matrix EffiencentNet 17
Figure 10: Learning Graph ResNet 17
Figure 11: Confusion Matrix ResNet 18
Figure 12:Learning Graph MobileNet 18
Figure 13:Confusion Matrix MobileNet 19
Figure 14:Successful route configuration with valid origin, destination, and A* pathfinding algorithm 20
Figure 15: Input validation prevents route calculation when origin and destination are identical. 21
Figure 16: Baseline route visualization generated using the A* search algorithm without traffic incidents 22
Figure 17: Route recalculation and avoidance behaviour under a severe traffic incident 22
Figure 18: AI-based incident severity classification result using a pre-trained ResNet18 model. 23

Instruction

System requirement

Before running the system, the following requirements should be met:

- Python version 3.9 or above
- A stable internet connection (required for loading map tiles)
- Windows, macOS, or Linux operating system
- A modern web browser such as Google Chrome or Microsoft Edge

Installation

First, download or extract all project files into a single folder on your computer. Open a terminal or command prompt and navigate to the project directory.

Install the required libraries by running:

pip install -r requirements.txt

This command installs all necessary dependencies, including libraries for machine learning, graph search, and map visualization.

Run the program

To start the system, run the following command:

python START_GUI.py

This launcher script automatically starts the application and loads the graphical user interface. After the program starts, a message will appear in the terminal indicating that the system is running. The interface can then be accessed by opening the following link in a web browser:

http://localhost:7860

Using the System Interface

The graphical user interface is designed to guide users through the route planning process in a step-by-step manner.

Step 1: Route Setup

Users begin by selecting an origin and a destination from the list of available heritage locations. The system checks the input to prevent invalid selections, such as choosing the same location for both the starting point and the destination.

Step 2: Traffic Incident Detection (Optional)

Users may upload an image showing a traffic incident. The system will analyse the image and classify the incident severity as None, Minor, Moderate, or Severe.
If no image is provided, users can still report an incident manually by selecting the affected road segment from a dropdown list.

Step 3: Algorithm Selection

Next, users select a pathfinding algorithm, such as Breadth-First Search, Depth-First Search, Uniform Cost Search, or A* Search. Brief information about the selected algorithm is displayed to help users understand how it works.

Step 4: Route Calculation and Visualization

After all selections are made, the system calculates the safest route and displays it on an interactive map. The route can be explored using playback controls that allow users to move forward or backward through each step of the algorithm. This helps users understand how the selected algorithm explores the map and reaches the destination.

5. Additional Notes

- If no custom map files are uploaded, the system uses a default heritage map of Kuching.
- Accident severity affects the travel cost of roads, which may result in different routes being selected.
- Multiple incidents can be applied at the same time.
- All machine learning predictions are performed locally and do not require an internet connection.

Introduction

Traffic congestion and road incidents are common challenges in urban environments and often result in delays, increased travel risk, and inefficient routing. Most conventional navigation systems prioritise the shortest or fastest path but do not adequately consider road safety conditions such as accidents, blockages, or hazardous areas. As a result, users may still be guided through unsafe routes despite the availability of alternative paths.

The SafeRoute Navigator was developed to address this limitation by integrating machine learning–based traffic incident detection with graph-based route planning algorithms. The system aims to recommend safer routes by dynamically adjusting route costs based on detected incident severity. Instead of relying solely on distance or travel time, the system incorporates safety as an additional decision factor.

This project combines artificial intelligence techniques for image-based incident severity classification with classical pathfinding algorithms such as A* and Uniform Cost Search. By integrating these components into an interactive web-based interface, the system demonstrates how AI can be applied to enhance traditional route planning and support safer navigation decisions in real-world traffic scenarios.

Features

Route Setup and Configuration

The system allows users to select an origin and destination from a predefined list of heritage locations in Kuching. Input validation is applied to prevent invalid configurations, such as selecting the same location as both the starting point and destination. Users are also given the option to upload custom map data; if no custom data is provided, the system automatically uses a default heritage map.

Multiple pathfinding algorithms are supported, including Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), Greedy Best-First Search, and A* Search. To support usability, a short algorithm guide is included within the interface to help users understand the basic differences and intended use cases of each algorithm.

(Refer to Appendix A.1)

Traffic Incident Reporting and Safety Analysis

The system provides two different methods for reporting traffic incidents. The first method allows users to manually select a road segment where an incident has occurred. The second method uses an AI-based approach, where users upload an image of a traffic incident and the system automatically classifies its severity.

Once an incident is reported, it is stored internally and visually marked on the map. The severity level of the incident, classified as None, Minor, Moderate, or Severe, directly affects the travel cost of the associated road segment. This ensures that safety conditions are incorporated into the route selection process.

(Refer to Appendix A.2)

AI-Based Incident Classification

The system integrates multiple convolutional neural network models for traffic incident severity classification, including ResNet18, MobileNetV3, and EfficientNetV2. Users can select the preferred model through the interface when performing AI-based classification.

After analysis, the predicted severity level, confidence score, and probability distribution across all severity classes are displayed to the user. This feature demonstrates the integration of machine learning into the routing system and enables comparison between different deep learning models in terms of prediction behaviour and confidence.

(Refer to Appendix A.3)

Route Calculation and Visualization

After route configuration and safety analysis, the system calculates the optimal route based on the selected algorithm and current road conditions. The resulting route is displayed on an interactive map generated using OpenStreetMap data. Start and destination points, route paths, and reported incident locations are clearly highlighted.

In addition, the system provides route analytics, including estimated travel time, total distance, number of waypoints, and the algorithm used during route calculation. These analytics allow users to better understand how routing decisions are made.

(Refer to Appendix A.4)

Step-by-Step Route Animation

The SafeRoute Navigator includes an interactive animation feature that allows users to explore how a selected algorithm finds a route. Playback controls enable users to move forward and backward through each step, pause the animation, or restart the visualization from the beginning.

This feature helps users visualize and understand the behaviour of different search algorithms, making the routing process more transparent and easier to interpret.

(Refer to Appendix A.5)

Bugs

During testing, several minor issues were identified. Small delays may occur when rendering large maps or when using fast animation playback. Incident markers may occasionally overlap depending on the map zoom level. In addition, OpenStreetMap data snapping may cause minor visual inaccuracies in the displayed route paths.

These issues do not affect the correctness of the route calculation and are primarily related to visual rendering.

Missing or Not Implemented Features

All required features specified in the assignment have been successfully implemented. However, several potential enhancements were not included in the current version. These include real-time traffic data integration, live CCTV camera feed support, a dedicated mobile application version, and support for multi-user concurrent access.

The missing features listed above are considered future enhancements rather than core system requirements.

Testing

Testing was conducted to verify the correctness, usability, and reliability of the SafeRoute Navigator system. The test cases focused on validating route configuration, pathfinding behavior, safety analysis, AI-based incident classification, and route visualization. A total of five representative test cases were designed to cover the core system functionalities under different scenarios. Screenshots and detailed outputs for each test case are provided in Appendix C.

Test Case 1: Valid Route Configuration and Successful Route Calculation

This test case verifies that the system correctly handles a valid route configuration. The user selects Masjid Bandaraya Kuching as the starting point and Plaza Merdeka as the destination, with the A* algorithm selected for pathfinding.

The system successfully validates the input, confirms that the origin and destination are different, and proceeds to route calculation. The calculated route is displayed on the map along with route analytics such as estimated travel time, number of waypoints, and algorithm used. This test confirms that the basic routing workflow operates as expected.

(Refer to Appendix C.1)

Test Case 2: Input Validation for Invalid Route Configuration

This test case evaluates the system's input validation mechanism. The user selects the same location, Masjid Bandaraya Kuching, as both the starting point and destination.

The system correctly detects the invalid configuration and prevents the user from proceeding. A warning message is displayed, indicating that the origin and destination cannot be the same. This confirms that input validation rules are enforced to prevent logical errors during route calculation.

(Refer to Appendix C.2)

Test Case 3: Route Visualization and Step-by-Step Animation

This test case focuses on the route visualization and animation features. After a successful route calculation, the user initiates the step-by-step animation to observe how the selected algorithm explores the route.

The system displays the route progressively, with playback controls allowing the user to play, pause, step forward, and step backward through the routing process. The animation completes correctly and reflects the expected behaviour of the selected algorithm. This test confirms that the visualization and animation controls function properly.

(Refer to Appendix C.3)

Test Case 4: Manual Traffic Incident Reporting and Route Impact

This test case evaluates the manual traffic incident reporting feature. The user manually reports an incident on a selected road segment and assigns a severity level.

The incident is successfully recorded and displayed on the map as an incident marker. The reported severity affects the route cost, causing the routing algorithm to avoid heavily impacted roads where applicable. This confirms that manually reported incidents are correctly integrated into the safety analysis and routing process.

(Refer to Appendix C.4)

Test Case 5: AI-Based Incident Classification and Severity Detection

This test case verifies the AI-powered incident classification feature. The user uploads an image of a traffic accident and selects a pre-trained model (ResNet18 V2) for classification.

The system successfully processes the image and outputs the predicted severity level, confidence score, and probability distribution across all severity classes. In this test, the incident is classified as Severe, with a high confidence score. The classification result is displayed clearly in the interface and can be used to influence route selection. This test confirms that the AI model integration and result presentation function as intended.

(Refer to Appendix C.5)

Testing Summary

Overall, the testing results indicate that the SafeRoute Navigator system functions correctly across its core features. Route setup, validation, pathfinding, safety analysis, AI-based classification, and visualization all performed as expected. Minor visual limitations were observed during animation playback and marker overlap, but these did not affect system correctness.

The five test cases collectively demonstrate that the system meets the functional requirements specified for this assignment.

Insights

Three pre-trained CNN models were evaluated for traffic incident severity classification: ResNet18, MobileNetV3, and EfficientNetV2. All models were pre-trained on the ImageNet-1K dataset and fine-tuned using the project dataset.

Among the models tested, **ResNet18** achieved the most balanced performance with a test accuracy of approximately **75%**. It showed stable predictions across severity levels, with minor confusion between *Minor* and *Moderate* cases. **MobileNetV3** prioritises fast and lightweight inference but produced lower accuracy (**~68%**), particularly for *Severe* incidents. **EfficientNetV2** demonstrated strong feature extraction but was more sensitive to class imbalance, resulting in slightly inconsistent predictions (**~65–70%**).

Based on these results, **ResNet18 was selected as the primary model** for integration into the SafeRoute Navigator system due to its balance between accuracy and reliability.

Model	Pretrained	Test Accuracy	Strengths	Limitations
ResNet 18	ImageNet-1K	≈75%	Stable performance, good generalization	Slight confusion between Minor & Moderate
MobileNetV3	ImageNet-1K	≈68%	Fast inference, lightweight	Lower accuracy on Severe cases
EfficientNetV2	ImageNet-1K	≈65–70%	Strong feature extraction	More sensitive to class imbalance

Conclusion

The SafeRoute Navigator system demonstrates how artificial intelligence and path-finding algorithms can be combined to support safer route planning in urban environments. By integrating classical search algorithms with AI-based traffic incident classification, the system is able to adjust route decisions based on detected road conditions and incident severity.

Through model evaluation, ResNet18 was selected as the most suitable incident classification model due to its balanced accuracy and stable performance across severity levels. The testing results show that AI-based severity detection can effectively influence route selection, allowing the system to avoid high-risk road segments when necessary. In addition, the step-by-step route visualization helps users understand how different algorithms explore and construct paths.

While the current system functions well using static map data and simulated incidents, future improvements could include real-time traffic data integration, live camera inputs, and support for mobile platforms. Overall, this project highlights the practical application of AI in intelligent transport systems and provides a solid foundation for further enhancements.

Appendix

Appendix A – User Interface

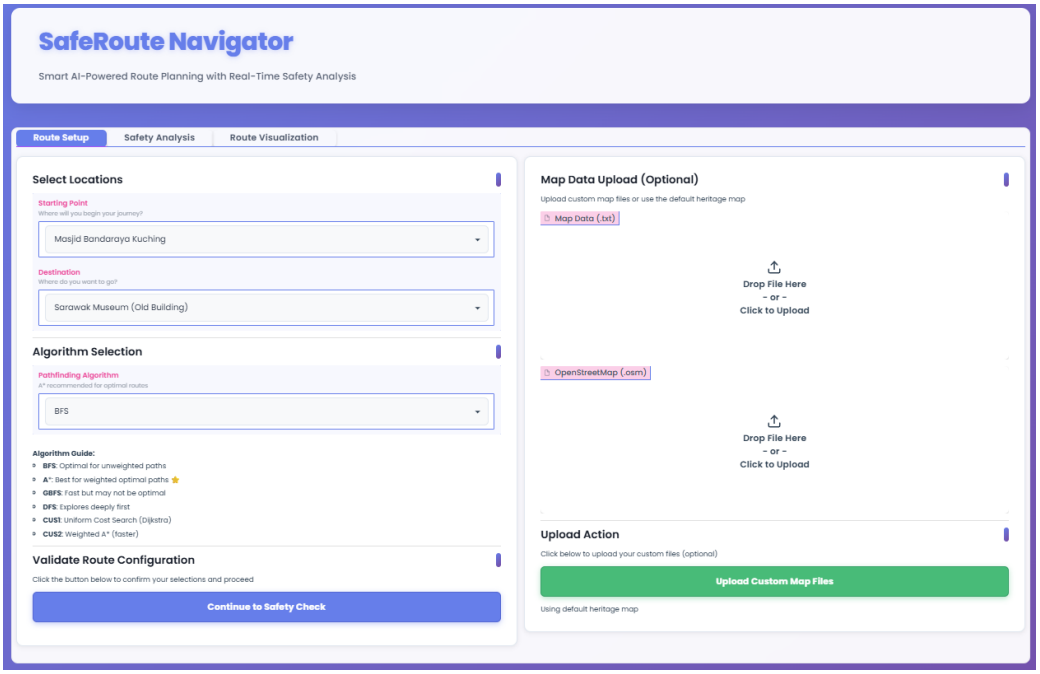

Figure 1:Route Setup Interface

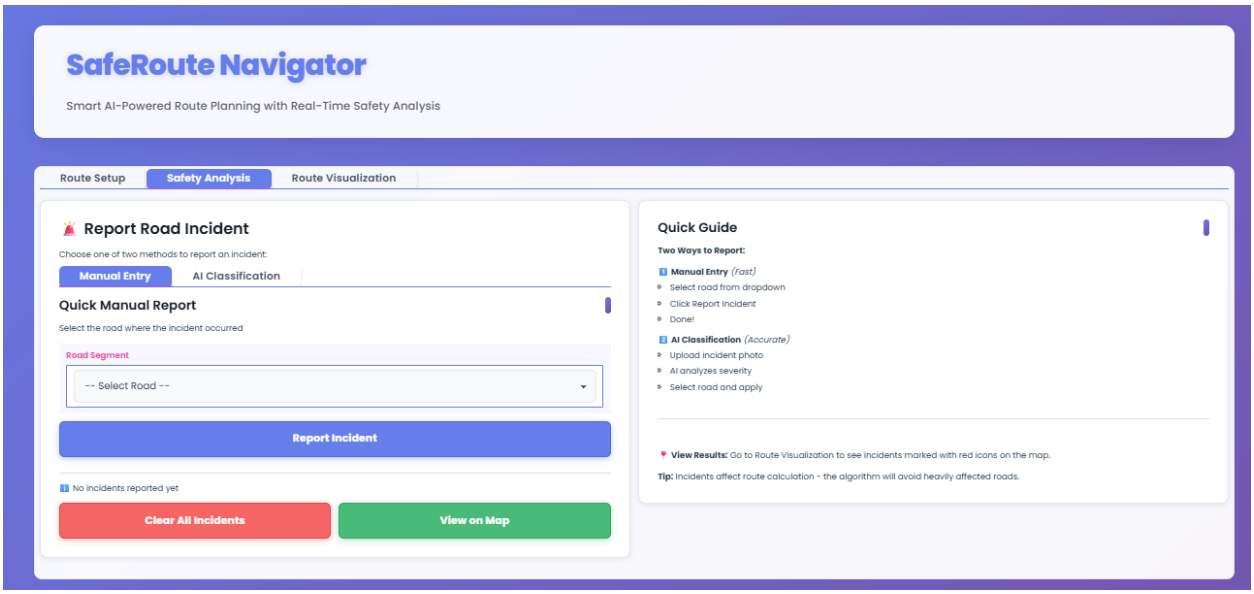

Figure 2:Safety Analysis – Manual Incident Reporting

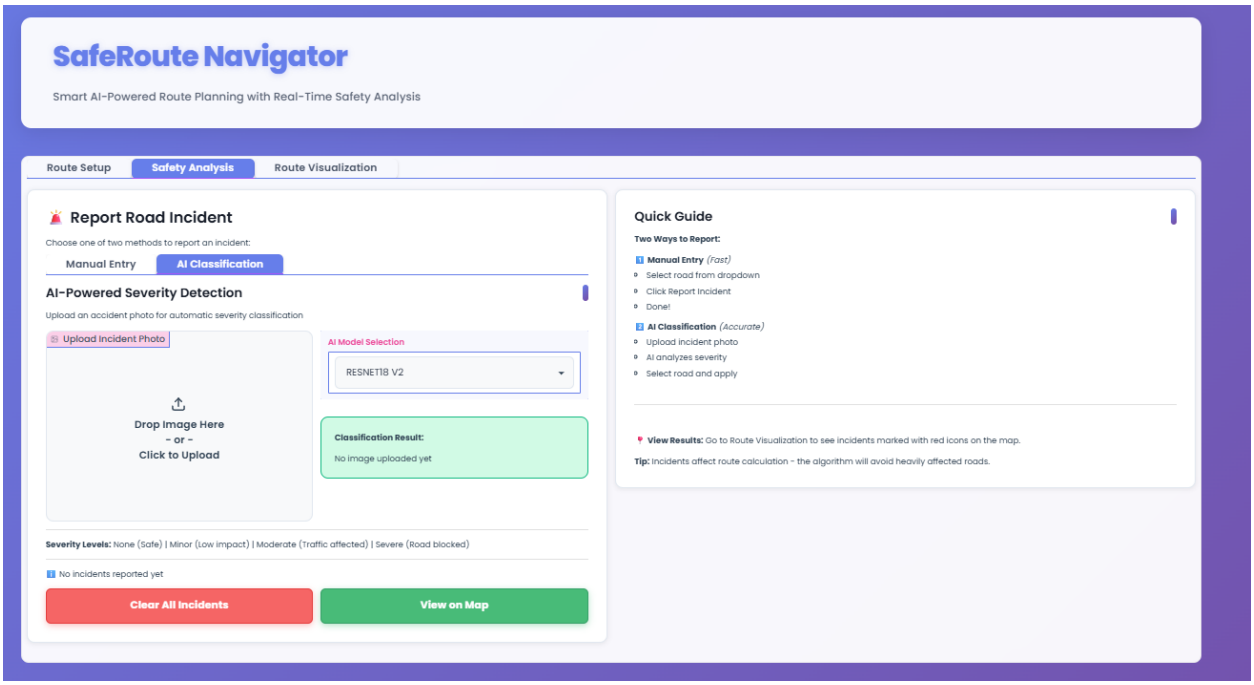

Figure 3: AI Incident Classification Interface

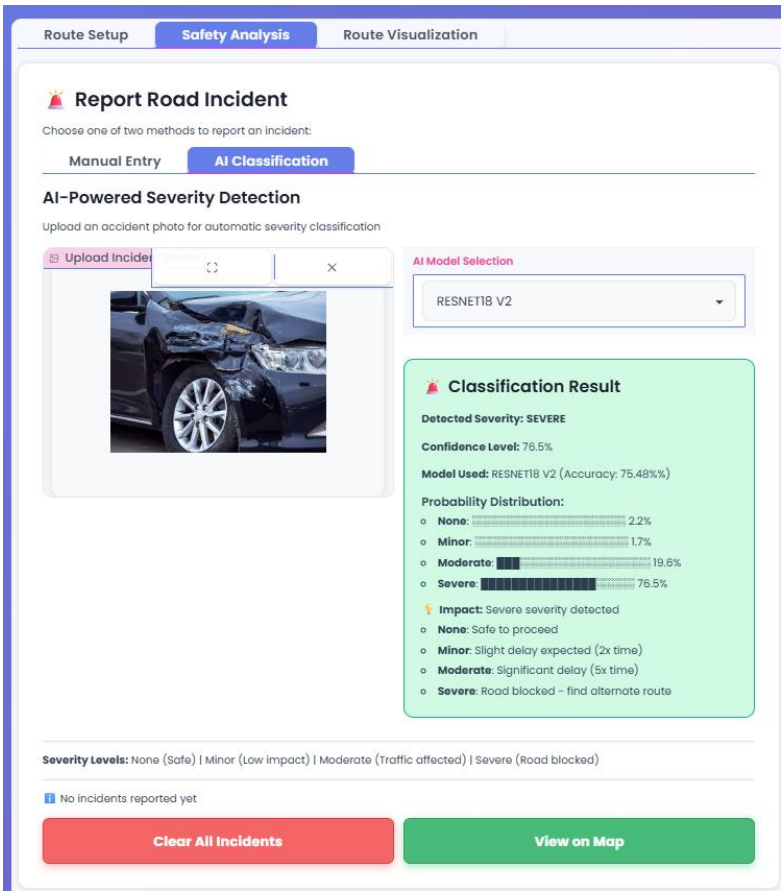

Figure 4: AI Incident Classification Result Display

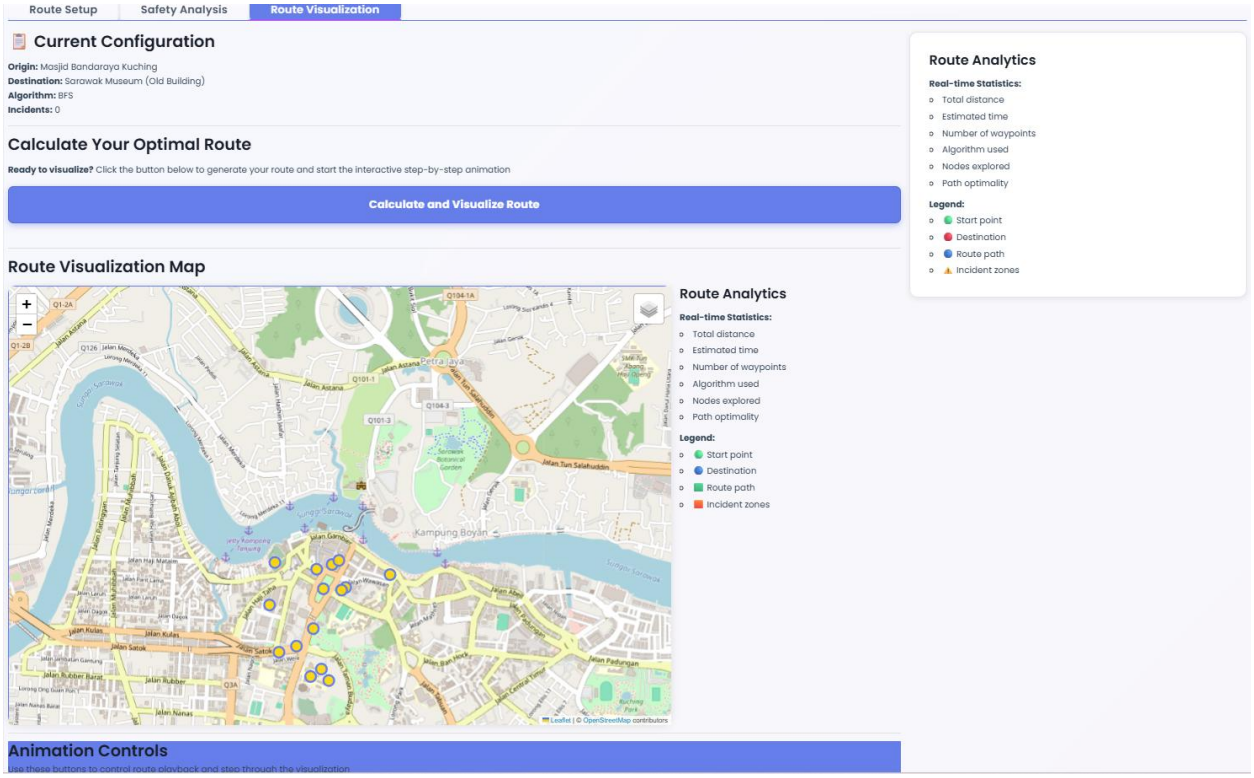

Figure 5: Route Calculation and Visualization Interface

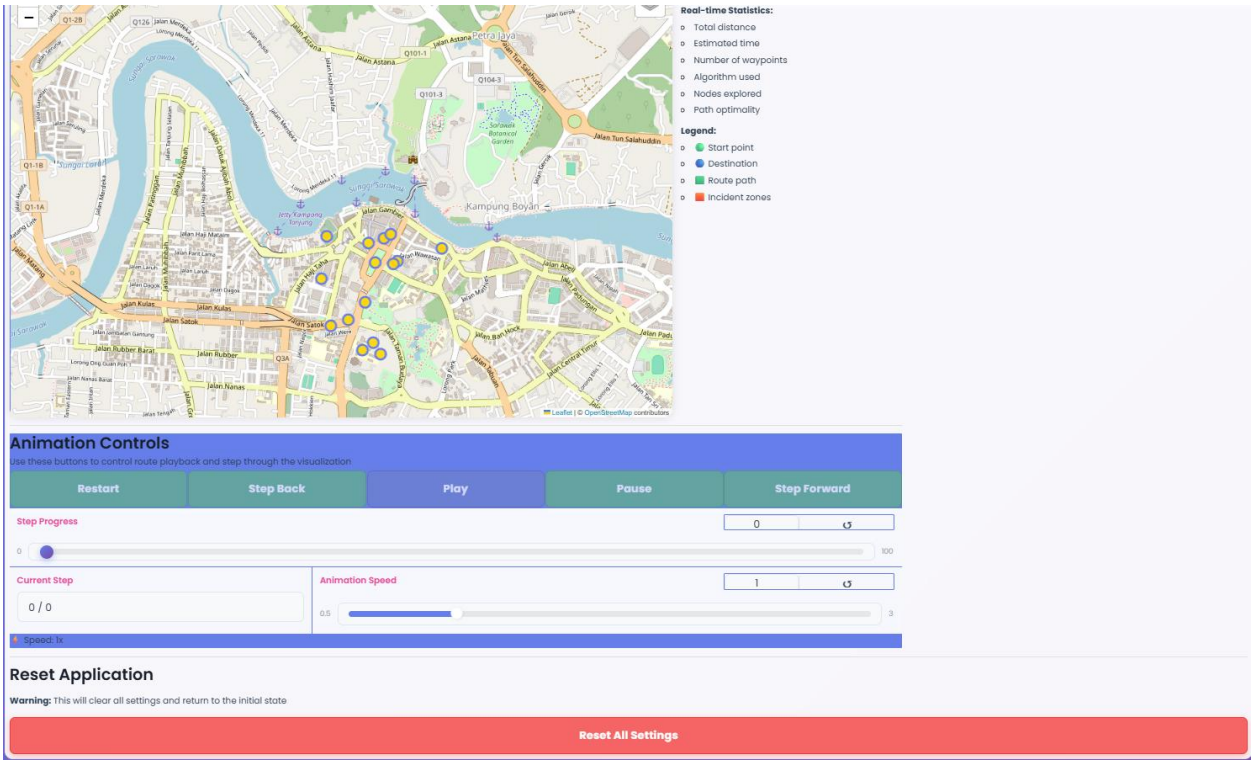

Figure 6: Route Playback and Animation Controls

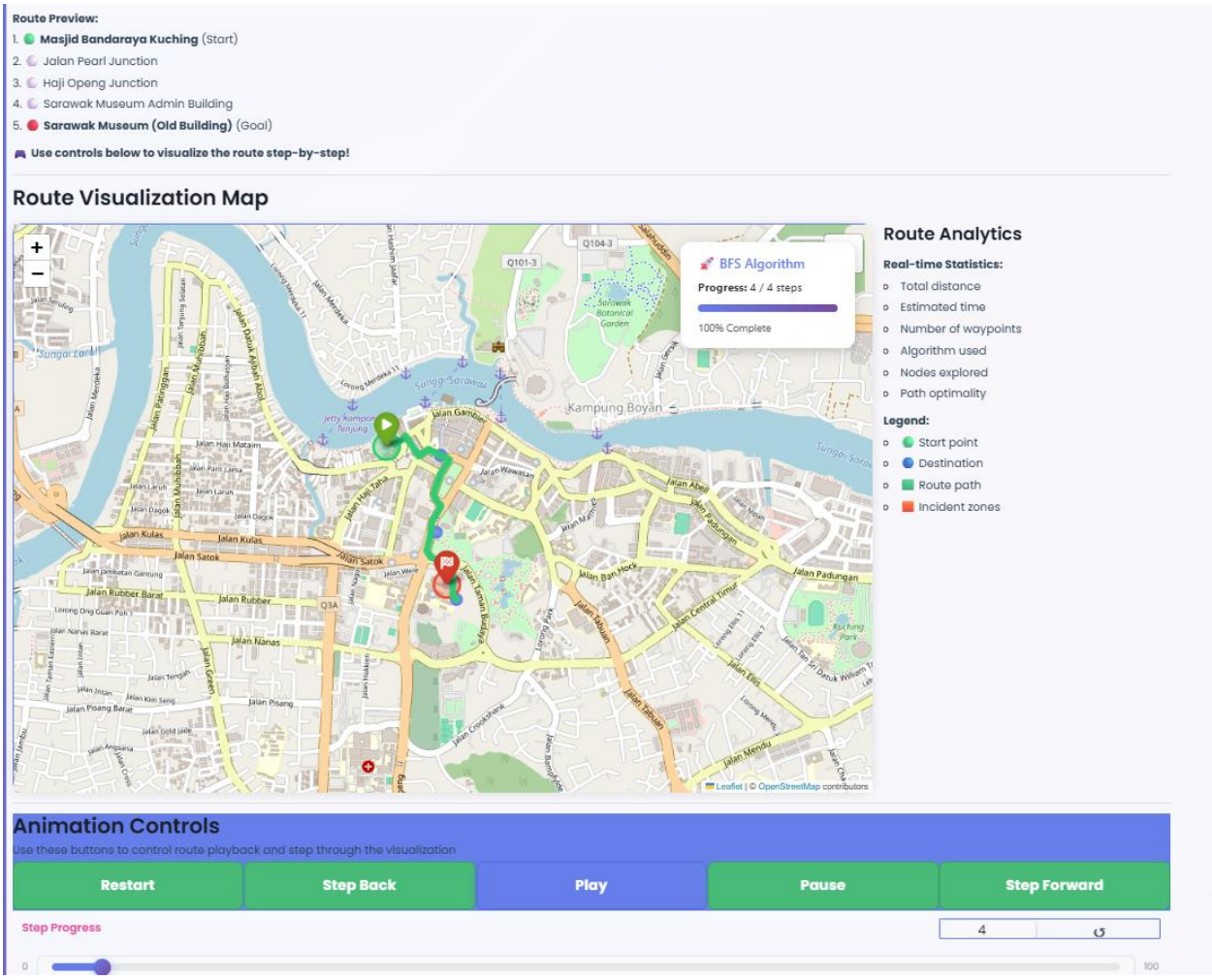

Figure 7:Calculated Route Preview and Visualization

Appendix B- AI Model Training

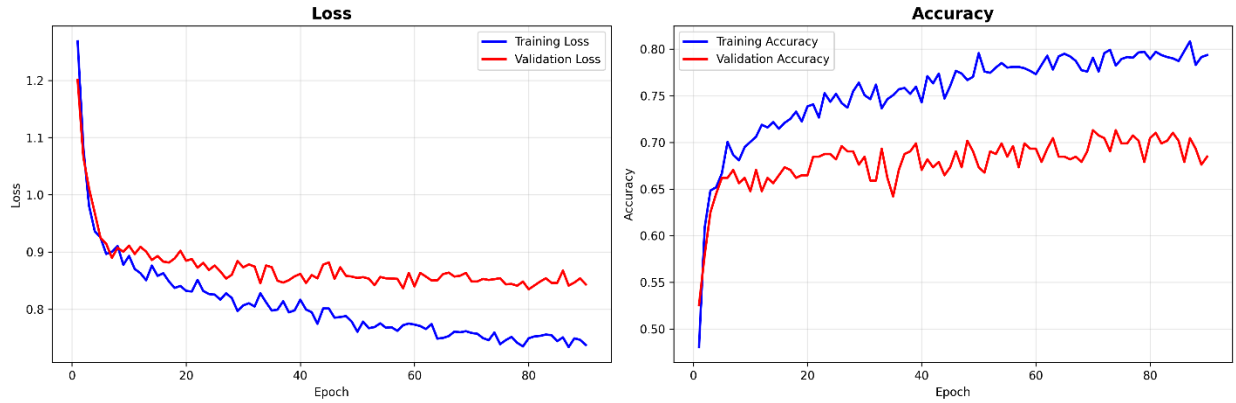

Figure 8:Learning Graph EfficientNet

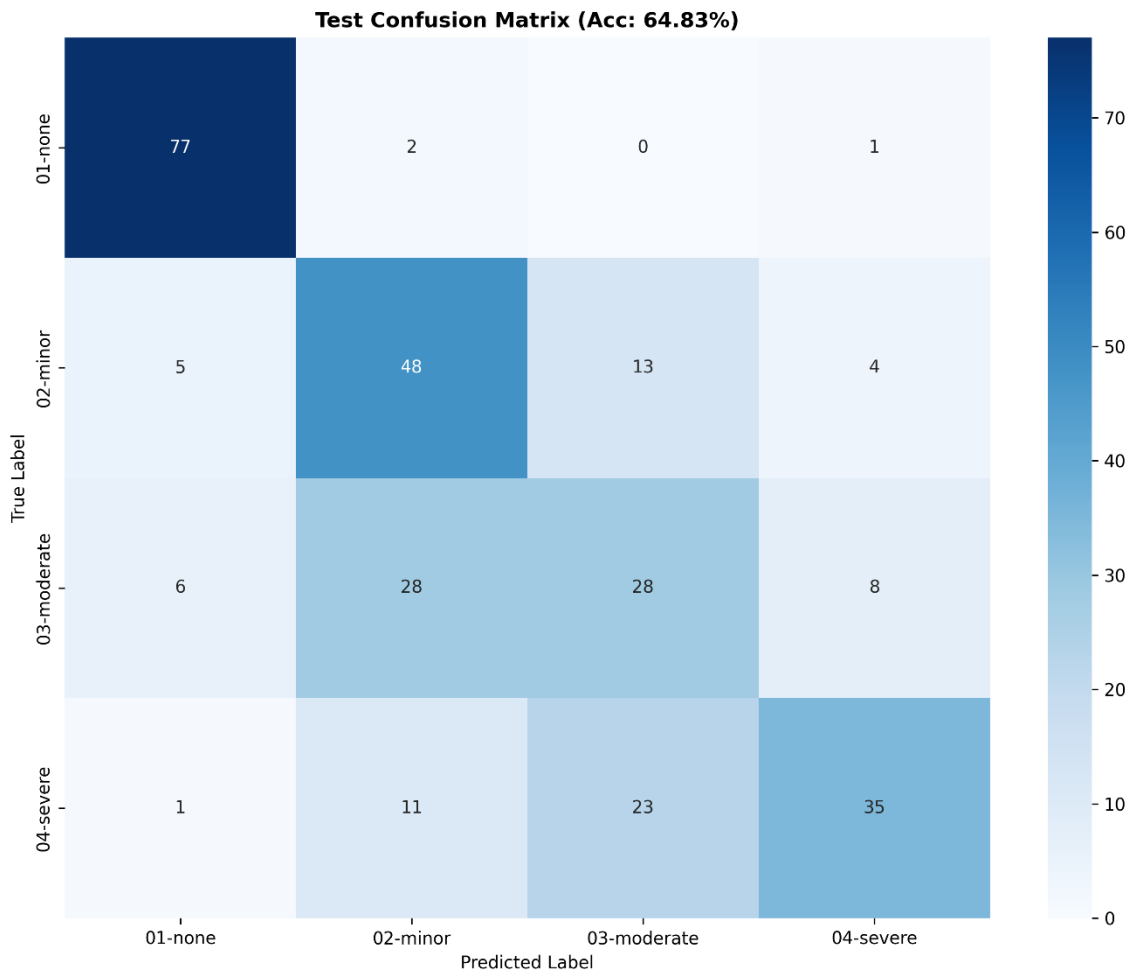

Figure 9:Confusion Matrix EffiencentNet

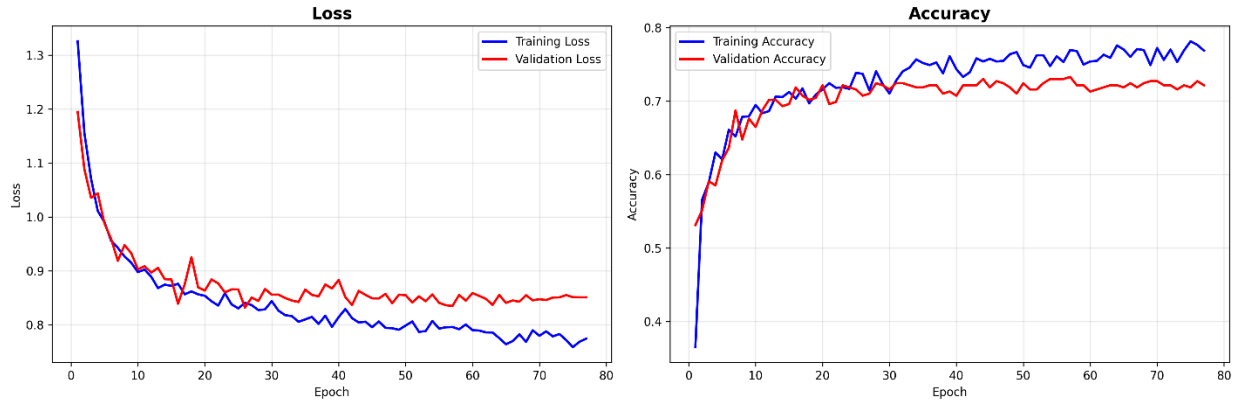

Figure 10: Learning Graph ResNet

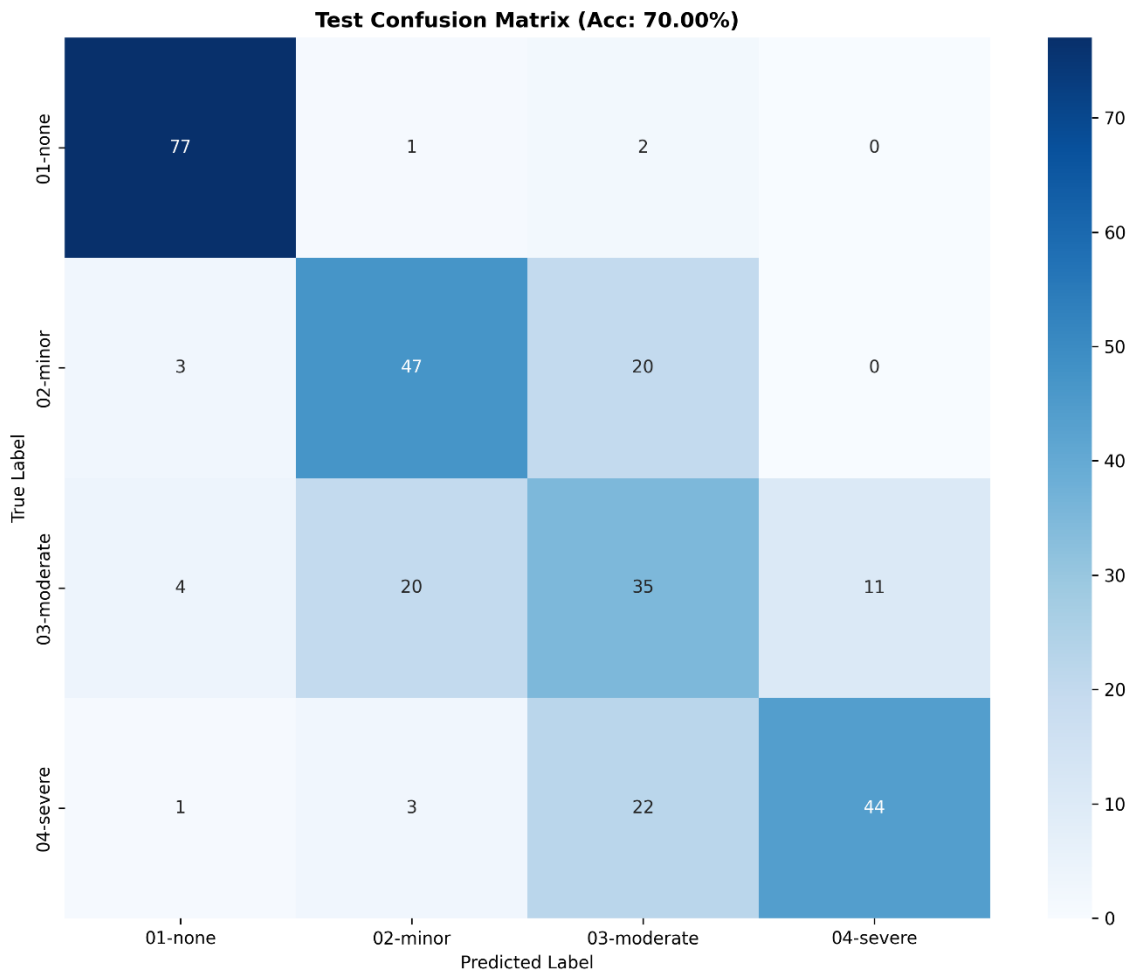

Figure 11: Confusion Matrix ResNet

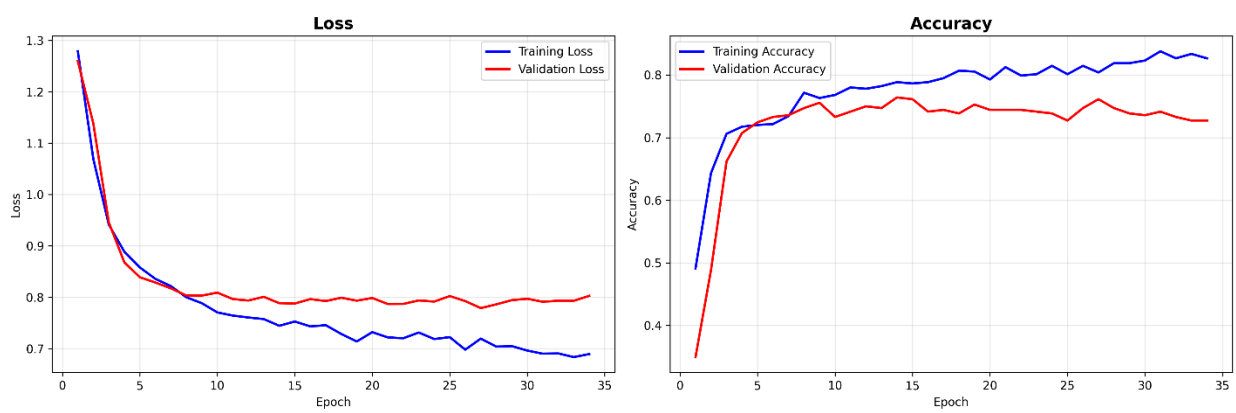

Figure 12:Learning Graph MobileNet

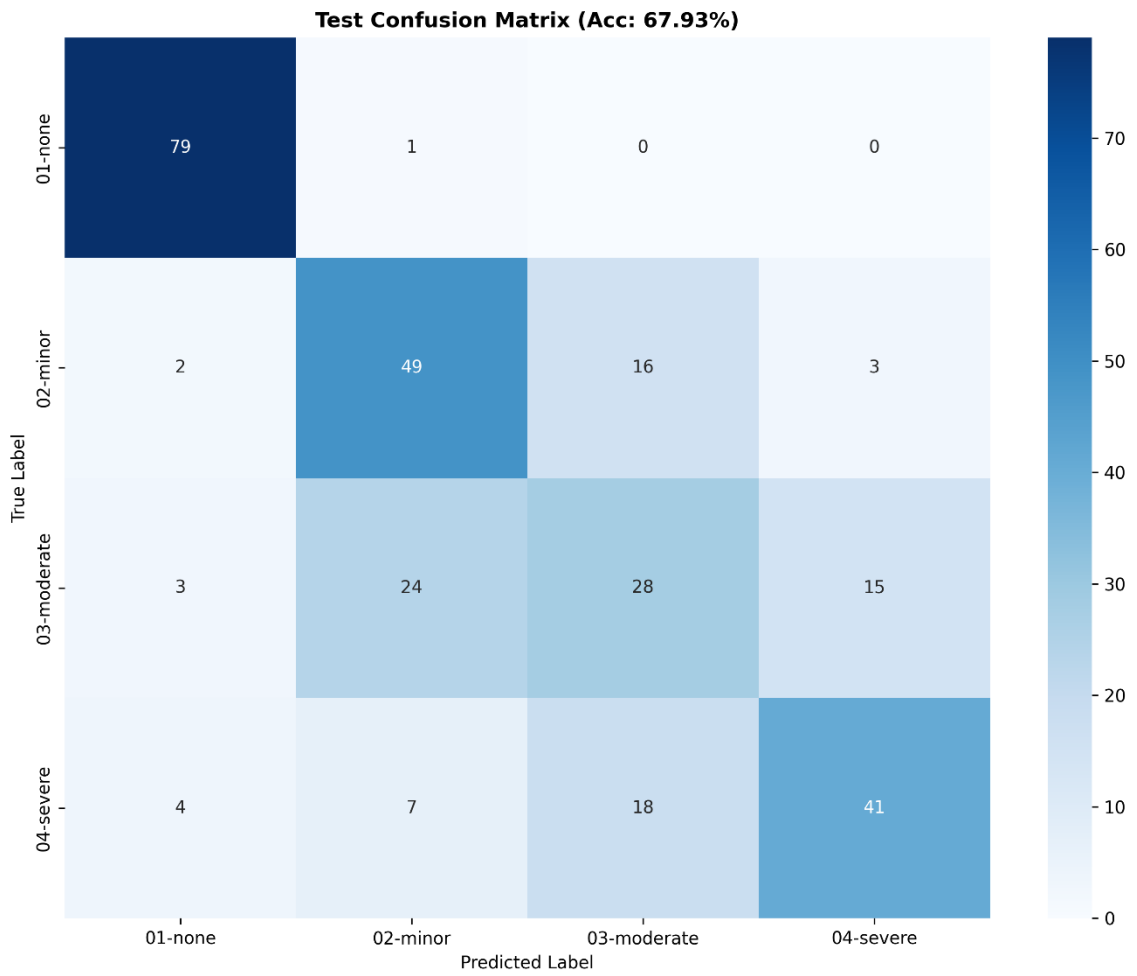

Figure 13:Confusion Matrix MobileNet

Appendix C- Testing

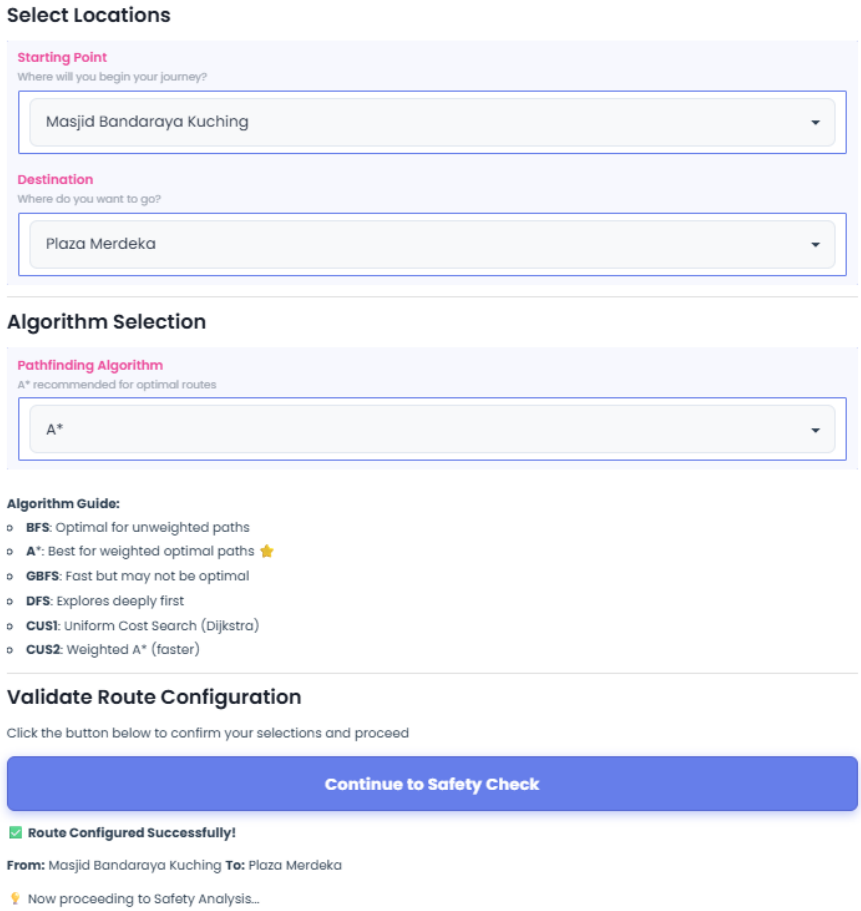

Figure 14:Successful route configuration with valid origin, destination, and A pathfinding algorithm.*

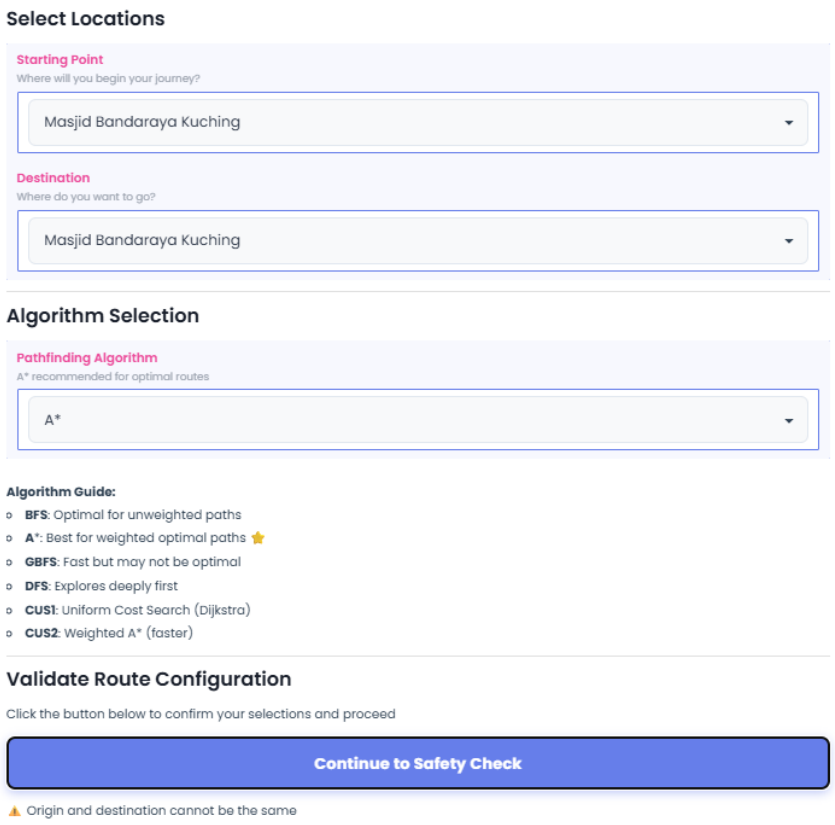

Figure 15: Input validation prevents route calculation when origin and destination are identical.

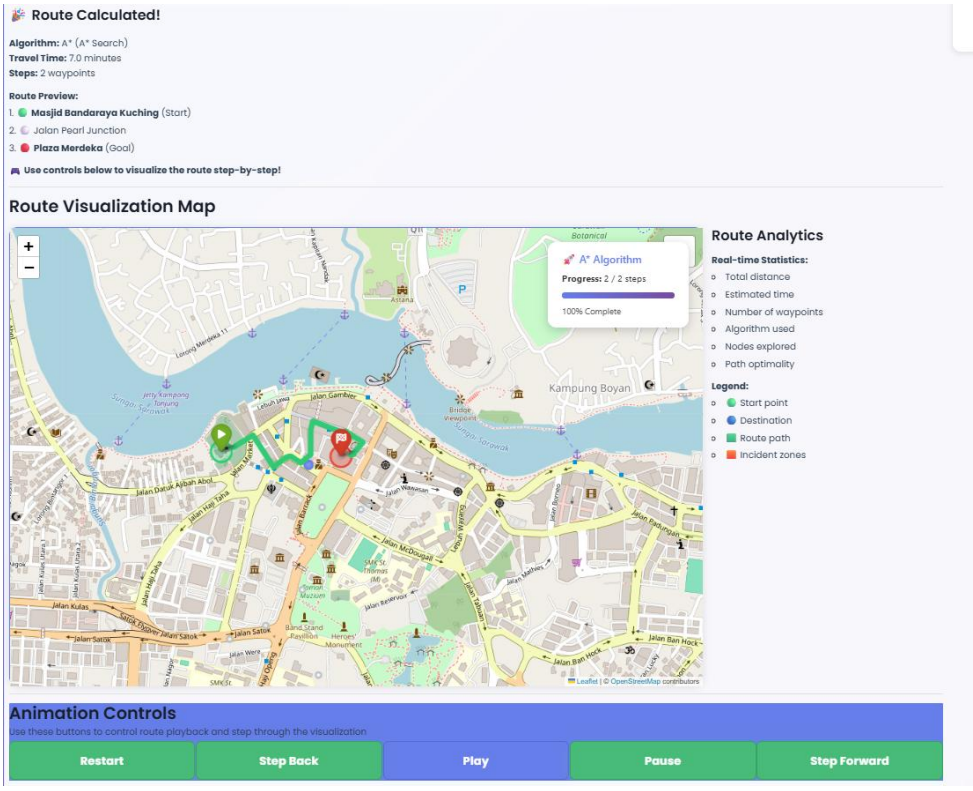

Figure 16: Baseline route visualization generated using the A search algorithm without traffic incidents.*

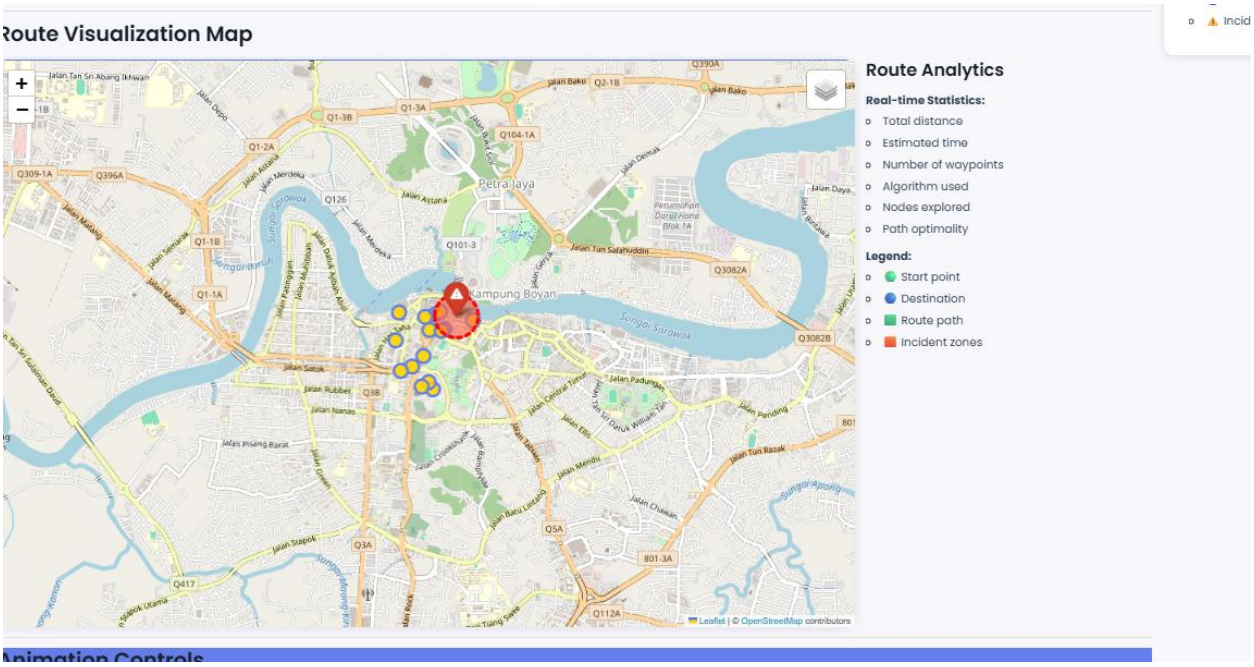

Figure 17: Route recalculation and avoidance behaviour under a severe traffic incident.

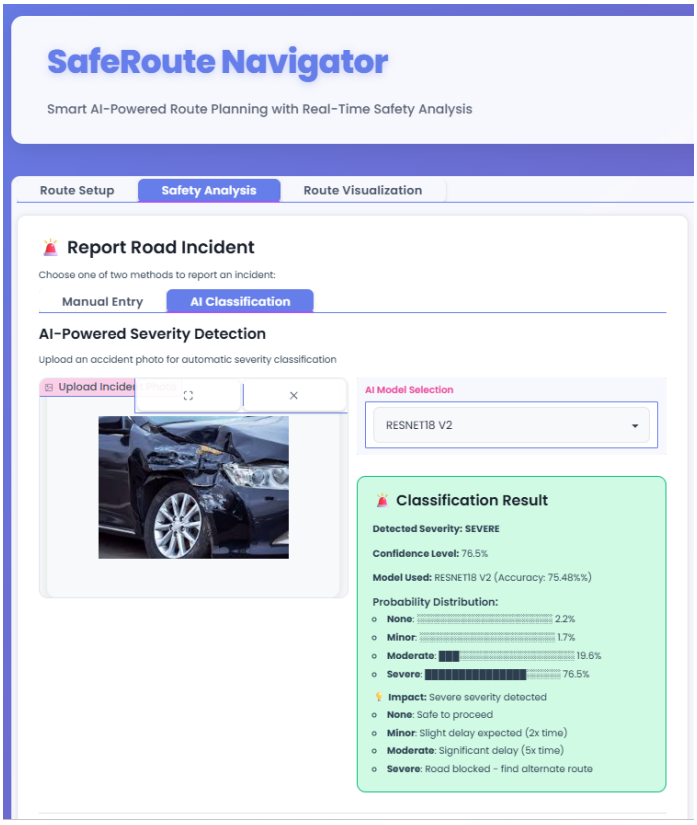

Figure 18: AI-based incident severity classification result using a pre-trained ResNet18 model.

Acknowledgements

- **PyTorch** – used for training and evaluating convolutional neural network models for traffic incident classification.
- **Torchvision** – used for pre-trained ImageNet models and image preprocessing.
- **Gradio** – used to develop the graphical user interface for interaction and testing.
- **NetworkX** – used to model the road network and implement pathfinding algorithms.
- **Folium** – used for interactive map visualisation and route display.
- **OpenStreetMap** – used as the base map data source for route planning and visualisation.
- **AI-assisted tools (e.g., ChatGPT)** – used to **assist in this assignment** by improving grammar, clarifying explanations, and helping with reference formatting. All system design, implementation, testing, and final decisions were completed by the authors.

References

[1] P. Bhamere, "Car Damage Severity Dataset," *Kaggle*, 2023. [Online]. Available: https://www.kaggle.com/datasets/prajwalbhamere/car-damage-severity-dataset. Accessed: Jan. 2025.

[2] A. Boukhris, "Cars Body Type Cropped Dataset," *Kaggle*, 2022. [Online]. Available: https://www.kaggle.com/datasets/ademboukhris/cars-body-type-cropped/data. Accessed: Jan. 2025.